

Revo 2 Basic Hand Setup and MuJoCo Validation

From model files to a verified six-actuator simulation

Cesar A. Pascal Vasquez

Manual 1

Purpose of this Manual

This manual explains how to prepare a Python environment, identify the BrainCo Revo 2 Basic MuJoCo assets, run the right and left hands, test the six active controls, and complete a repeatable headless validation. The experiment is intentionally limited to the hand model. It establishes a trusted baseline before adding ball-policy training or mounting the hand on the Unitree R1.

Experiment goal

At the end of the experiment, both hand models should load without missing meshes, execute open, power, and pinch commands, and complete 1,000 headless simulation steps with finite joint positions.

Contents

1	Before You Begin	3
1.1	Obtain the model files	3
1.2	What you will produce	3
2	Project Overview	4
2.1	Software and files used	4
2.2	Experiment boundary	4
3	Recommended Folder Structure	5
3.1	Preflight checklist	5
4	Create the Python Environment	6
5	Inspect the Hand Model	7
6	Understand the Six Active Controls	8
7	Choose Parameters and Write a Controller	9
8	Baseline Code Tour	10
9	Exactly Where to Change Parameters	11
10	Copy, Edit, Run, and Compare	12
11	Run the Interactive Simulations	13
12	Test Poses and Individual Actuators	14
13	Run the Headless Validation	15
14	Formal Experiment Procedure	16
15	Troubleshooting	17
15.1	The viewer does not open	17
15.2	A mesh is missing	17
15.3	A finger moves backward	17
15.4	The ball jumps away	17
15.5	macOS blocks a launcher	17
16	Final Checklist and Conclusion	18

1 Before You Begin

This manual is independent: no earlier experiment is required. Work only in simulation. You need a macOS, Linux, or Windows computer with Python 3.10 or newer, at least 1 GB of free space, and a graphics driver that supports OpenGL 3.3. The headless checks can run without opening a viewer.

1.1 Obtain the model files

Open the BrainCo Revo 2 download center listed in the references and obtain the MuJoCo/XML package. Extract `Revo2_xml.zip`. Confirm that it contains `xml_right/`, `xml_left/`, and their `meshes/` folders. Do not continue if the mesh files are missing.

The source package contains `editable_code/revo2_sim.py`. Copy it to a local `scripts/` folder before beginning the exercise.

1.2 What you will produce

By the end, your local project will contain a working environment, a right-hand and left-hand validation result, a custom six-value pose, and a short record of the parameters you chose.

Checkpoint 1

Before installing Python packages, you should be able to point to both hand XML files, both mesh folders, and `scripts/revo2_sim.py`. Record their local paths in your laboratory notes.

Your design decision

Choose the hand you will modify first: _____. Choose one behavior to design: power grasp, pinch, wave, or your own pose: _____.

2 Project Overview

The Revo 2 Basic hand has five fingers and six active controls. The thumb uses two controls: opposition and flexion. The index, middle, ring, and little fingers each use one proximal flexion control. Additional distal joints are present in the model, but their motion is mechanically coupled rather than commanded independently.

This distinction is important when planning learning experiments. A controller sends six commands per hand, not one command for every visible joint. The simulation wrapper converts normalized commands into the official actuator ranges contained in the BrainCo XML model.

2.1 Software and files used

- Python 3 and a local virtual environment;
- MuJoCo 3.3.6 or newer within major version 3;
- NumPy 2.x;
- BrainCo right- and left-hand XML/STL assets;
- `scripts/revo2_sim.py` for interactive and headless tests.

2.2 Experiment boundary

The wrist remains fixed, gravity is disabled in the first viewer demonstration, and the sphere begins near the palm. This is a geometry and actuation test, not a complete grasping benchmark and not physical-robot control.

Basic versus Touch

Blue pad geometries participate in simulated contact. They must not be interpreted as measurements from physical tactile sensors. The Revo 2 Basic hand does not include the tactile modules of the Touch edition.

3 Recommended Folder Structure

Keep the official vendor assets separate from the wrapper scripts. A clean project should resemble the following structure.

Project tree

```
brainco_revo2_mujoco/  
|-- requirements.txt  
|-- README.md  
|-- Launch Revo2 Simulation.command  
|-- Launch Left Hand.command  
|-- scripts/  
|   |-- revo2_sim.py  
|-- vendor/  
|   |-- Revo2_xml.zip  
|   |-- Revo2_xml/  
|       |-- xml_right/  
|           |-- brainco-righthand-v2.xml  
|           |-- scene_grasp.xml  
|           |-- meshes/  
|       |-- xml_left/  
|           |-- brainco-lefthand-v2.xml  
|           |-- scene_grasp.xml  
|           |-- meshes/  
|-- policies/  
|-- reports/
```

Do not rename the mesh folders without also updating the XML asset paths. Relative asset paths keep the project portable between computers.

3.1 Preflight checklist

- ☐ The XML files exist for both hands.
- ☐ The corresponding `meshes/` folders contain STL files.
- ☐ The project path contains no partially extracted ZIP archive.
- ☐ At least 1 GB of free disk space is available for the virtual environment.

4 Create the Python Environment

Open Terminal, change into the project directory, and create a local environment. Using `.venv` prevents the experiment from changing packages used by unrelated projects.

One-time setup

```
cd "/path/to/brainco_revo2_mujoco"
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt
```

The requirements file contains only the packages needed by the base simulator.

```
1 mujoco>=3.3.6,<4
2 numpy>=2.0,<3
```

Listing 1: Minimal package requirements

Confirm the installation before opening a graphics window.

Version check

```
.venv/bin/python -c "import mujoco, numpy; print(mujoco.__version__); print(numpy.__version__)"
```

Expected result

The command should print one MuJoCo version and one NumPy version without an import error. Package warnings are acceptable if the program continues, but an exception must be resolved before proceeding.

5 Inspect the Hand Model

MuJoCo models are defined with MJCF XML. The scene file includes the hand model, a camera, lighting, a floor or background, and the grasp object. The official XML then references the STL meshes that produce the visible hand.

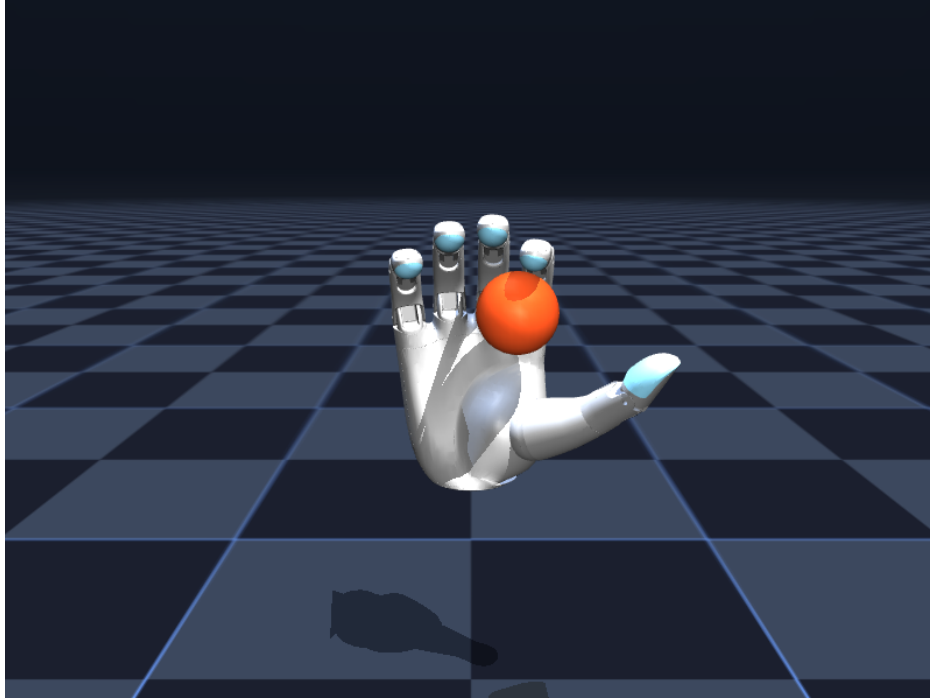


Figure 1: Right Revo 2 Basic hand in the standalone MuJoCo scene.

Use a short Python inspection command to confirm model size and actuator names.

```
1 from pathlib import Path
2 import mujoco
3
4 scene = Path("vendor/Revo2_xml/xml_right/scene_grasp.xml")
5 model = mujoco.MjModel.from_xml_path(str(scene))
6
7 print("bodies:", model.nbody)
8 print("joints:", model.njnt)
9 print("actuators:", model.nu)
10 for actuator_id in range(model.nu):
11     print(mujoco.mj_id2name(
12         model, mujoco.mjtObj.mjOBJ_ACTUATOR, actuator_id
13     ))
```

Listing 2: Loading the right-hand scene and listing actuators

If MuJoCo reports a missing mesh, verify the spelling and capitalization of every path in the XML. macOS may tolerate some naming differences that fail on Linux or another build system.

6 Understand the Six Active Controls

The wrapper addresses the following actuator suffixes for either the right or left hand.

Table 1: Normalized Revo 2 Basic control channels

Channel	Control	Primary effect
1	Thumb opposition	Moves the thumb toward or away from the palm
2	Thumb flexion	Curls the thumb
3	Index flexion	Curls the index finger
4	Middle flexion	Curls the middle finger
5	Ring flexion	Curls the ring finger
6	Little-finger flexion	Curls the little finger

The experiment expresses each command as a ratio from 0 to 1 and converts that ratio into the actuator’s XML range.

```
1 def ratio_to_ctrl(ratio, ctrl_low, ctrl_high):  
2     ratio = np.clip(ratio, 0.0, 1.0)  
3     return ctrl_low + ratio * (ctrl_high - ctrl_low)
```

Listing 3: Mapping normalized commands into MuJoCo control ranges

Three reference poses are included: open, power grasp, and pinch. The power grasp ratios are approximately [0.82, 0.88, 0.84, 0.88, 0.90, 0.92]. Apply commands gradually; instantly closing collision meshes can create unrealistic impulses.

7 Choose Parameters and Write a Controller

The supplied wrapper is a starting point. Copy it before editing, then change one variable at a time. The ranges below are practical simulation study ranges, not physical-hand safety limits.

Table 2: Student-controlled variables

Variable	Start	Study range	Main effect
Hand	right	right/left	Selects mirrored XML and actuator names
Pose ratios	supplied	0.0–1.0	Changes six finger commands
Closure duration	1.5 s	0.5–3.0 s	Controls contact speed and impulse
Physics step	XML value	0.0005–0.004 s	Trades speed for numerical resolution
Gravity z	0	0 to -9.81 m/s^2	Changes whether the fixed hand must support weight
Run length	1,000	500–5,000 steps	Changes validation duration

Create `scripts/my_controller.py`. The minimum interface is shown below; replace `MY_POSE` and `CLOSE_SECONDS` with your choices.

```

1 import numpy as np
2 from revo2_sim import Revo2Simulation
3
4 MY_POSE = np.array([0.75, 0.82, 0.70, 0.74, 0.76, 0.78])
5 CLOSE_SECONDS = 1.5
6
7 sim = Revo2Simulation("right")
8 steps = int(CLOSE_SECONDS / sim.model.opt.timestep)
9 for k in range(steps):
10     alpha = (k + 1) / steps
11     sim.target_ratio[:] = alpha * MY_POSE
12     sim.step(k * sim.model.opt.timestep)
13
14 assert np.all(np.isfinite(sim.data.qpos))
15 print("custom controller passed", sim.target_ratio)

```

Listing 4: A student-written pose controller

Record your design

Pose ratios: _____ Closure duration: _____ s. Predict which finger will contact first and explain why before running the code.

8 Baseline Code Tour

The complete baseline is `editable_code/revo2_sim.py`. Copy it to the local project's `scripts/` folder. Read it from top to bottom once before editing. It is organized into six parts.

Table 3: How the baseline file is organized

Order	Search for	Responsibility
1	<code>SCENES</code>	Maps <code>right/left</code> to the scene XML
2	<code>ACTUATOR_SUFFIXES</code>	Declares the six actuator names in command order
3	<code>POSES</code>	Stores open, power, and pinch ratios
4	<code>parse_args</code>	Reads <code>-hand</code> , <code>-headless</code> , and <code>-steps</code>
5	<code>class Revo2Simulation</code>	Loads MuJoCo, maps ratios, resets, and steps physics
6	<code>run_headless/run_interactive</code>	Chooses validation or viewer execution

The runtime data flow is:

1. `main()` parses the terminal arguments.
2. `Revo2Simulation(hand)` loads one XML and finds six actuators by name.
3. A pose or keyboard event updates `target_ratio` in the interval 0–1.
4. `ratio_to_ctrl()` converts ratios into the XML actuator ranges.
5. `update_controls()` slew-limits the command.
6. `step()` calls `mujoco.mj_step()` once.
7. The headless path checks finite joint positions; the interactive path synchronizes the viewer.

Preserve the interface

The order of `ACTUATOR_SUFFIXES`, each pose array, and the six actuator IDs must agree. If you reorder one without the others, the numbers will drive the wrong fingers.

9 Exactly Where to Change Parameters

Open `scripts/revo2_sim.py` in any text editor. Use Find to locate the symbol in the second column; line numbers are not used because they change when students add comments.

Table 4: Parameter-to-code map for the baseline

Parameter	Find this symbol	Default	What to edit
Scene paths	SCENES	vendor paths	Change only if the XML folders differ
Pose ratios	POSES	three arrays	Edit six values, each from 0.0 to 1.0
Gravity	GRAVITY_Z	0.0	Use -9.81 for Earth gravity
Key increment	KEY_INCREMENT	0.05	Smaller values provide finer manual control
Auto cycle	AUTO_CYCLE_SECONDS	5.0 s	Period of the open–close demonstration
Closing time	CLOSE_SECONDS	5.0 s	Slew limit for a full-range command
Headless length	--steps	1,000	Supply a positive terminal argument
Hand side	--hand	right	Supply <code>right</code> or <code>left</code>
Camera	viewer.cam	supplied	Change look-at, distance, azimuth, elevation
Physics step	XML option	vendor value	Edit the MJCF only after baseline validation

The four uppercase student parameters appear immediately below `POSES`. They were separated from the class so students can change behavior without rewriting the simulation logic.

```

1 GRAVITY_Z = 0.0
2 KEY_INCREMENT = 0.05
3 AUTO_CYCLE_SECONDS = 5.0
4 CLOSE_SECONDS = 5.0

```

Listing 5: The editable parameter block in the supplied baseline

Controlled change

Select one parameter from the parameter-to-code map. Keep every other value fixed. Baseline value: _____. New value: _____. Expected observation: _____.

10 Copy, Edit, Run, and Compare

Never edit the only copy of the baseline. Create a student version and verify that the unchanged copy still passes.

Create an editable experiment

```
cp scripts/revo2_sim.py scripts/my_revo2_experiment.py
.venv/bin/python scripts/revo2_sim.py \
  --hand right --headless --steps 1000
```

In `my_revo2_experiment.py`, add a fourth pose inside `POSES`. Keep exactly six entries.

```
1 POSES["gentle"] = np.array([.62, .68, .55, .58, .60, .62])
2
3 # Inside key_callback(), add before the reset branch:
4 elif key == "G":
5     self.set_pose("gentle")
```

Listing 6: Example student pose and keyboard binding

Run the student version first without gravity, then with one chosen parameter change. For headless validation, both runs must finish with finite joint positions.

Run the edited code

```
.venv/bin/python scripts/my_revo2_experiment.py --hand right
.venv/bin/python scripts/my_revo2_experiment.py \
  --hand right --headless --steps 2000
```

Table 5: Baseline-versus-edit record

Observation	Baseline	Edited code
Parameter value		
First contacting finger		
Visible oscillation or impulse		
Finite 2,000-step result		
Unexpected behavior		

If the edited version fails, restore the last working value and repeat with a smaller change. This baseline–edit–compare loop is the required experimental method for this manual.

11 Run the Interactive Simulations

The easiest right-hand launch command is:

Right hand

```
cd "/path/to/brainco_revo2_mujoco"  
zsh "./Launch Revo2 Simulation.command"
```

Run the left-hand mirror model separately.

Left hand

```
zsh "./Launch Left Hand.command"
```

After setup, the direct Python forms are:

Direct Python execution

```
.venv/bin/python scripts/revo2_sim.py --hand right  
.venv/bin/python scripts/revo2_sim.py --hand left
```

Click the MuJoCo window before using the keyboard. Verify that the camera shows the palm and that the fingers do not begin inside the ball or another mesh.

Expected result

Both viewers should open, show mirrored hand geometry, and remain numerically stable while the hand is open. Closing the viewer should return control to Terminal without a Python traceback.

12 Test Poses and Individual Actuators

Use the keyboard to test high-level poses first, then isolate each actuator.

Table 6: Interactive keyboard controls

Key	Action
O	Open the hand
C	Apply the power grasp
P	Apply the pinch pose
A or Space	Toggle the automatic open-close demonstration
1-6	Select one active control
[and]	Decrease or increase the selected control
R	Reset the model and object
Escape	Exit

For each hand, complete this sequence:

1. Press **O** and confirm that all fingers extend.
2. Press **C** and observe a coordinated power grasp.
3. Press **P** and verify that the thumb and index form the main pinch.
4. Select channels 1 through 6 and change each by one increment.
5. Press **R** and confirm that the open pose returns.

Record reversed, frozen, or excessively fast channels. A sign or control-range error must be fixed before training.

13 Run the Headless Validation

The headless test removes camera and desktop-window issues from the diagnosis. It loads the model, enables an automatic grasp cycle, performs a fixed number of physics steps, and checks that every joint position remains finite.

Right-hand validation

```
.venv/bin/python scripts/revo2_sim.py \  
  --hand right --headless --steps 1000
```

Left-hand validation

```
.venv/bin/python scripts/revo2_sim.py \  
  --hand left --headless --steps 1000
```

The expected terminal line has the following form.

Expected output

```
Validation passed: hand=right, actuators=6, joints=11, bodies=..., steps=1000
```

Do not ignore numerical warnings

An exploding object, NaN, non-finite joint value, or MuJoCo instability warning is a failed validation even if the process exits. Check mesh intersections, time step, control ranges, and command slew rate.

14 Formal Experiment Procedure

Use the same procedure whenever the hand XML, meshes, or actuator mapping changes.

1. Record the project identifier and MuJoCo version.
2. Run the right-hand headless test with 1,000 steps.
3. Run the left-hand headless test with 1,000 steps.
4. Open the right-hand viewer and test open, power, and pinch poses.
5. Repeat the pose test for the left hand.
6. Change each of the six controls separately and record its direction.
7. Reset the simulation five times and watch for inconsistent object motion.
8. Save one screenshot of the correctly rendered hand.

Table 7: Suggested experiment record

Test	Observation	Pass criterion
Right headless		1,000 stable steps
Left headless		1,000 stable steps
Right poses		Three recognizable poses
Left poses		Three recognizable poses
Individual channels		Six correct movement directions
Repeated reset		No launch impulse or mesh overlap

15 Troubleshooting

15.1 The viewer does not open

Run the headless test first. If it passes, the model is valid and the remaining problem is likely the macOS graphics session or MuJoCo viewer access. Launch from an interactive Terminal rather than a background process.

15.2 A mesh is missing

Confirm that the XML's mesh directory is relative to the XML location and that the STL filename matches exactly. Re-extract the vendor archive if a file has zero bytes.

15.3 A finger moves backward

Inspect the actuator control range and the joint axis. Do not compensate by reversing training labels until the physical joint convention has been confirmed.

15.4 The ball jumps away

Reset to the open pose, reduce closure speed, and check for initial mesh penetration. The wrapper already slew-limits controls because a full command in one physics step can create an unrealistic impulse.

15.5 macOS blocks a launcher

Control-click the `.command` file, choose Open, and approve it once. The direct Python command remains available if Finder launch permissions are inconvenient.

16 Final Checklist and Conclusion

- ☐ A local `.venv` exists and imports MuJoCo and NumPy.
- ☐ Both official hand XML files load with all meshes.
- ☐ The model reports six actuators per hand.
- ☐ Open, power, and pinch poses are recognizable.
- ☐ Individual channels 1 through 6 move the expected digit.
- ☐ Both headless tests complete 1,000 finite steps.
- ☐ The experiment remains explicitly simulation-only.

This experiment produces a verified low-level hand simulation. The next manual adds gravity, randomized ball placement, disturbances, quantitative success metrics, and policy training. Keeping this validation separate makes later failures easier to diagnose: training should never be used to hide a broken model or incorrect actuator mapping.

References

1. BrainCo Hand. Revo 2 download center.
<https://www.brainco-hz.com/docs/revolimb-hand-dev/en/revo2/download.html>
2. BrainCo Hand. Revo 2 product parameters.
<https://www.brainco-hz.com/docs/revolimb-hand-dev/en/revo2/parameters.html>
3. DeepMind. MuJoCo documentation.
<https://mujoco.readthedocs.io/en/stable/overview.html>
4. DeepMind. MuJoCo XML reference.
<https://mujoco.readthedocs.io/en/stable/XMLreference.html>